

```
#include "FibonacciSequence.h"
```

```
// Constructor - Initializes the sequence with by setting fPrevious to 0 and fCurrent to 1. ➤
```

```
FibonacciSequence::FibonacciSequence() noexcept  
    : fPrevious(0), fCurrent(1)  
{  
}
```

```
// Operator* - Returns a constant reference to the current Fibonacci value.
```

```
const uint64_t& FibonacciSequence::operator*() const noexcept {  
    return fCurrent;  
}
```

```
// Prefix Increment operator - Advances the Fibonacci sequence.
```

```
FibonacciSequence& FibonacciSequence::operator++() noexcept {  
    uint64_t temp = fCurrent;           // Save the current value  
    fCurrent = fCurrent + fPrevious;    // Calculate the next Fibonacci number  
    fPrevious = temp;                  // Update fPrevious to the old current value  
    return *this;                      // Return the updated object  
}
```

```
// Postfix increment operator
```

```
FibonacciSequence FibonacciSequence::operator++(int) noexcept {  
    FibonacciSequence temp = *this;    // Saves the current state  
    ++(*this);                        // Advances the iterator  
    return temp;                      // Returns the old state  
}
```

```
// Equality operator - Checks if two FibonacciSequence objects are equal by comparing their fPrevious and fCurrent values. ➤
```

```
bool FibonacciSequence::operator==(const FibonacciSequence& other) const ➤  
    noexcept {  
    return (fPrevious == other.fPrevious) && (fCurrent == other.fCurrent);  
}
```

```
// begin - Resets the sequence to its starting values (0 and 1)
```

```
void FibonacciSequence::begin() noexcept {  
    fPrevious = 0;  
    fCurrent = 1;  
}
```

```
// end() - Sets the sequence to a state (fPrevious and fCurrent are zero) that does not occur in valid Fibonacci generation. ➤
```

```
void FibonacciSequence::end() noexcept {  
    fPrevious = 0;  
    fCurrent = 0;  
}
```